

Day 11 - Working With Real Data: JSON, CSV, HTTP APIs, Virtual Environments

Day 11 — Working With Real Data: JSON, CSV, HTTP APIs, Virtual Environments

Objectives

- Get the full, proper treatment of JSON (you used a small slice of it already, back on Day 7)
- Read and write CSV (spreadsheet-style) files correctly, using Python's `csv` module rather than parsing them by hand
- Make real HTTP requests to fetch data from the internet, and handle the many ways that can fail
- Understand what a virtual environment is and why practically every real Python project uses one

JSON, properly this time

Back on Day 7, you used just enough JSON to save and load your task tracker's data. Today you get the complete picture. **JSON** (JavaScript Object Notation) is a plain-text format for representing data — lists, dicts (called "objects" in JSON's own terminology), strings, numbers, `true/false`, and `null` — that's understood by virtually every programming language, which is exactly why it's the standard way computer programs (including web servers, called **APIs** — more on those below) send structured data to each other over the internet.

Python's built-in `json` module converts between JSON text and Python's own equivalent data types:

JSON	Python
object { }	<code>dict</code>
array []	<code>list</code>
string	<code>str</code>
number	<code>int</code> or <code>float</code>
<code>true / false</code>	<code>True / False</code>
<code>null</code>	<code>None</code>

```
import json
```

```
data = {"name": "Ana", "tags": ["admin", "active"], "active": True}
```

```
json_text = json.dumps(data, indent=2)    # convert a Python object INTO a JSON string (note the "s")
python_object = json.loads(json_text)     # convert a JSON string BACK INTO a Python object (also
```

```

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)                # write directly to a FILE (no "s" -- operates on a fi

with open("data.json") as f:
    loaded = json.load(f)                       # read directly FROM a file (also no "s")

```

Notice the naming pattern precisely: `dump(s)` always means "Python → JSON" (writing out), and `load(s)` always means "JSON → Python" (reading in). The versions **with** an `s` (`dumps/loads`) work with plain strings sitting in memory; the versions **without** an `s` (`dump/load`) work directly with an already-open file object. `indent=2` simply makes the output nicely readable by a human, with line breaks and 2-space indentation, instead of one long, cramped line — it has no effect on what the data actually means, only on how it looks.

A genuine trap worth knowing about: JSON has no concept of a `tuple` or a `set` — only lists exist in JSON's world. `json.dumps` will silently convert any tuple you give it into a JSON array on the way out, but there's no way to tell, on the way back in, that it was ever supposed to be a tuple rather than a list — `json.loads` will always hand you back an ordinary `list`. If you specifically need a tuple or a set to survive a round trip through JSON unchanged, you have to convert it yourself, deliberately, both before saving and after loading.

CSV — for spreadsheet-style, row-and-column data

CSV stands for "Comma-Separated Values" — a simple text format for tabular data (rows and columns, like a basic spreadsheet), where each line is one row, and commas separate the individual column values within that row. You might be tempted to just use `.split(",")` from Day 5 to parse a CSV file yourself by hand — **don't**. That approach silently breaks the moment any single field itself happens to contain a comma (a very ordinary thing to happen, e.g. a name field containing "`Smith, John`"), because the file format has specific rules (quoting fields that contain commas) that a plain `.split(",")` doesn't know about at all. Python's built-in `csv` module already handles all of these rules correctly for you:

```

import csv

with open("people.csv", newline="", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)                # reads just the FIRST row (commonly the column names) and advan
    for row in reader:
        print(row)                       # each remaining row comes back as a plain list of strings

with open("people.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)           # automatically uses the very first row as column names/keys
    for row in reader:
        print(row["name"], row["age"])   # each row now comes back as a DICT, keyed by column name

with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "age"])     # write ONE row (here, acting as the header row)
    writer.writerows([["Ana", 30], ["Bo", 25]]) # write SEVERAL rows at once, from a list of lists

```

Two details specific to CSV that are easy to forget, but worth internalizing now: first, `newline=""` when opening a file for CSV work — this is required on some systems to stop Python's own automatic newline handling from interfering with the CSV module's own separate line-ending rules (a well-known, easy-to-forget quirk, so just always include it for CSV files). Second, and more conceptually important: **every single value read from a CSV file comes back as plain text (a string), even values that look exactly like numbers —**

the CSV format itself has no concept of "this column contains numbers." If you need an actual number to do arithmetic with, you must explicitly convert it yourself, exactly like you practiced with `parse_csv_line` back on Day 5 (`int(row["age"])`, for example).

HTTP requests — how your program talks to other computers over the internet

An **API** (Application Programming Interface) here refers to a way one program can ask another program — often running on a completely different computer somewhere else in the world — for data, or ask it to do something, over the internet. Communicating this way almost always uses **HTTP**, the same underlying protocol (a fancy word for "agreed-upon set of rules for communicating") your web browser uses every time it loads a web page. Python's standard library has a way to do this (`urllib`), but nearly the entire Python community instead uses a much friendlier third-party library called `requests`, which you install with:

```
pip install requests
```

(`pip` is Python's built-in tool for installing extra packages that don't come bundled with Python itself — you'll use it constantly from here on. "Third-party" just means "written and published by someone other than the core Python team," as opposed to `json` or `csv`, which are part of Python's own "standard library" and need no installation at all.)

```
import requests

response = requests.get("https://api.example.com/users/1", timeout=5)

response.status_code    # a number describing what happened: 200 (success), 404 (not found), 500 (s
response.json()         # parses the response body as JSON, handing you back a Python dict/list d
response.text           # the raw response body, as plain text, with no parsing applied
response.ok             # True if status_code is less than 400 (a rough, convenient "did it ba

response.raise_for_status() # if the status code indicates failure (4xx or 5xx), this line RAISES
```

Always include a `timeout`. Without one, if the other computer never responds at all (a common real-world failure — a server that's down, a bad network connection), your program will simply sit there frozen, waiting forever, with no way out. This is a genuine, well-known category of real production incident ("why did our program just hang?"), not a theoretical, only-in-textbooks concern.

Real code always assumes a network call *might* fail, and handles that possibility explicitly, using exactly the `try/except` pattern from Day 6:

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
except requests.RequestException as e:
    print(f"Request failed: {e}")
```

`requests.RequestException` is a broad category covering everything the `requests` library itself might raise — connection failures, timeouts, bad status codes after `raise_for_status()`. Catching it is a direct, practical application of the "EAFP" idea from Day 6 (attempt the operation, and be ready to handle it failing), applied specifically to network calls — which, unlike most code you've written so far, genuinely can fail at any moment for reasons entirely outside your own program's control.

Sending data **to** a server (rather than just requesting it) typically uses a POST request:

```
response = requests.post(
    "https://api.example.com/users",
    json={"name": "Ana", "age": 30},    # requests automatically converts this dict to JSON text and
    timeout=5,
)
```

Virtual environments — giving every project its own, separate set of installed packages

If you simply `pip install` packages without any further setup, they get installed **globally** — meaning every single Python project on your entire computer shares one common pool of installed packages and their exact versions. This becomes a real problem the moment you have two different projects that each need a **different** version of the same package — you genuinely cannot have two different versions of one package installed globally at the same time.

A **virtual environment** solves this by giving one specific project folder its own private, isolated, separate copy of installed packages, completely independent of anything installed globally or in any other project's own virtual environment.

```
python -m venv .venv    # create a new virtual environment, stored in a folder named .venv (d
```

Before you can actually use it, you must **activate** it — every single time you open a fresh terminal window and want to work on that project:

```
.venv\Scripts\activate    # Windows
source .venv/bin/activate  # macOS/Linux
```

Once activated, your terminal prompt usually changes to show the environment's name, and from that point on, anything you `pip install` goes into this project's own private `.venv` folder, not anywhere global:

```
pip install requests    # installs INTO .venv specifically, not system-wide
pip freeze > requirements.txt    # writes down the EXACT versions of everything currently installed
```

`requirements.txt` becomes a portable, shareable "shopping list" — on a different computer entirely (a teammate's laptop, or a server), running `pip install -r requirements.txt` (after creating and activating a fresh `.venv` there too) recreates that exact same set of packages, at those exact same versions, from scratch. This is **why** nearly every real Python project you'll ever open at a job includes a `requirements.txt` (or, in more modern projects, a `pyproject.toml` — you'll meet this on Day 13) and expects you to set up your own virtual environment before running anything at all. When you're done working, `deactivate` leaves the virtual environment and returns your terminal to its normal, global state.

Exercises

Exercises 1-4 (JSON and CSV) are auto-checked and don't need any internet connection — open `exercises.py`, fill in the `# TODOs`, and run `python exercises.py`. Exercise 5, the real HTTP API call, is a separate manual exercise described in `api_exercise.md` in this same folder, since it needs an internet connection and the `requests` package installed — read that file and follow its instructions once you've finished `exercises.py`.